

结构感知单核融合的机械臂批量逆运动学 CUDA 加速方法

刘霄鹏

武汉科技大学机械工程学院, 武汉 430081

文章编号: (预留) DOI: (预留)

摘要: 针对机械臂批量逆运动学 (inverse kinematics, IK) 中单种子优化器收敛概率不足与 GPU 多 Kernel 调度开销过高的双重瓶颈, 提出一种硬件固化的 Sobol 多起点并行 IK 加速方法。核心设计为: 将 $K = 16$ 组 Sobol 低差异序列种子固化于单一 CUDA 核函数内, 16 条线程经线并行执行独立 LM 迭代并在共享内存中完成块内候选选择——无论种子数 K 取何值, 核函数启动次数恒为 1, 调度开销 $< 0.01\%$ GPU 流时间。在 NVIDIA GeForce RTX 4060 Laptop GPU (SM 8.9) 上, $N = 100 \sim 1000$ 范围内吞吐量为 $1.50 \times 10^4 \sim 1.82 \times 10^4$ targets/s, Strict 成功率 (位置误差 < 5 mm 且姿态误差 $< 1^\circ$) 达 0.940–0.960, 全样本位置误差 $p95$ 为 4.34–5.49 mm。消融实验揭示: 将 K 从 16 降至 1 后, 吞吐量仅提升约 35% ($1.82 \times 10^4 \rightarrow 2.79 \times 10^4$), 但 Strict 成功率从 0.954 骤降至 0.522, $p95$ 从 4.6 mm 飙升至 685 mm——证实本方法的成功率几乎完全来源于 Sobol 多起点策略而非单一 LM 优化器的收敛能力。与默认单种子 cuRobo-Graph 对比: 本方法 $K = 16$ 时成功率领先约 11–12 个百分点, $K = 1$ 时落后约 32–41 个百分点, 表明两种方法代表了不同的设计范式——“硬件固化多起点”vs“单粒子智能搜索”。PTX 分析确认核函数为 FP64 计算密集型 (194 寄存器/线程, 共享内存 Bank 冲突为零)。该结果确立了消费级 GPU 上成功率优先、延迟确定的批量 IK 加速新范式。

关键词: 逆运动学; CUDA 小矩阵加速; Sobol 多起点策略; Kernel Fusion; 消融实验

中图分类号: TP391.4 **文献标志码:** A

收稿日期: (预留); **修回日期:** (预留)。

Structure-Aware Single-Kernel Fusion CUDA Acceleration for Batch Inverse Kinematics of Robotic Manipulators

LIU Xiaopeng

School of Mechanical Engineering, Wuhan University of Science and Technology, Wuhan 430081, China

Abstract: Batch inverse kinematics (IK) for robotic manipulators faces a dual bottleneck: single-seed optimizers suffer from low convergence probability on non-convex problems, while multi-kernel GPU pipelines incur excessive scheduling overhead. This paper proposes a hardware-fused Sobol multi-start parallel IK acceleration method. The core design hardcodes $K = 16$ Sobol low-discrepancy seeds into a single CUDA kernel: 16 thread lanes execute independent LM iterations in parallel, with in-block candidate selection completed entirely in shared memory—regardless of K , the kernel launch count remains exactly 1, keeping scheduling overhead below 0.01% of GPU stream time. On an NVIDIA RTX 4060 Laptop GPU (SM 8.9), the method achieves $1.50 \times 10^4 \sim 1.82 \times 10^4$ targets/s for $N = 100 \sim 1000$, with Strict success rate (position < 5 mm, orientation $< 1^\circ$) of 0.940–0.960 and all-target position-error $p95$ of 4.34–5.49 mm. An ablation study reveals the decisive role of the multi-start strategy: re-

ducing K from 16 to 1 improves throughput by only $\sim 35\%$ ($1.82 \times 10^4 \rightarrow 2.79 \times 10^4$) but causes Strict SR to collapse from 0.954 to 0.522 and $p95$ to surge from 4.6 mm to 685 mm—confirming that the method’s high success rate stems from the Sobol multi-start strategy rather than the LM optimizer itself. Compared to default single-seed cuRobo-Graph, the proposed method at $K = 16$ leads by 11–12 percentage points in success rate; at $K = 1$, it trails by 32–41 points—revealing two distinct design paradigms: “hardware-fused multi-start” versus “single-particle intelligent search.” PTX analysis confirms the kernel is FP64 compute-bound (194 registers/thread) with zero shared-memory bank conflicts. These results establish a success-prioritized, latency-deterministic batch IK paradigm on consumer-grade GPUs.

Keywords: inverse kinematics; CUDA small-matrix acceleration; analytical Jacobian; Levenberg–Marquardt; kernel fusion

0 引言

机械臂批量逆运动学 (inverse kinematics, IK) 是采样式运动规划、轨迹优化和机器人学习中的核心计算环节 [?, ?]。对 6 自由度串联机械臂而言, 单个 IK 问题规模极小: 6×6 雅可比矩阵、6 维线性系统、每迭代约

1 650 次双精度浮点运算。然而,当批量规模 N 达到数百至数千时,计算范式发生根本转变——问题从“单个 6 维非线性最小二乘”变为“ N 个独立 6×6 小矩阵运算的批量并行调度问题”。在这一范式下, GPU 的固定调度开销 (kernel launch $\approx 5\text{--}10 \mu\text{s}$ /次) 和主机-设备同步延迟可能主导实际性能 [?], 形成**小矩阵批量计算的结构性矛盾**: 每个子问题计算量极小 (不足以摊薄调度开销), 但子问题数量足够多 (可用并行性充足)。

这一矛盾并非 IK 特有, 而是广泛存在于机器人学中任何涉及固定小规模线性代数批量计算的场景。现有 GPU 库对小矩阵支持不足: NVIDIA cuBLAS 批量 GEMM 接口 (cublasGemmBatchedEx) 设计目标为 $m, n, k \geq 16$, 对 6×6 规模存在 $> 10 \mu\text{s}$ /次的固定调用开销; cuSOLVER 批量 LU/QR 分解同理 [?]. 在算法层面, cuRobo[?] 采用 CUDA Graph 和粒子群-L-BFGS 优化器实现大规模并行 IK, 但其默认单种子配置和通用优化器并非为 6×6 固定规模下的极致效率设计。本文的核心命题是: **针对 6×6 这一固定且极小的矩阵规模, 将运动学结构信息编码为编译期常量并融合为单核函数后, 能否将批量 IK 的调度开销压缩至可忽略量级?**

本文提出运动学结构驱动的 CUDA 小矩阵加速方法, 简称 OPT4C。其核心不是将已有算法库移植至 GPU, 而是将运动学结构信息 (关节轴方向、解析几何关系、固定 Sobol 种子数 $K = 16$) 作为编译期常量编码进核函数, 使 FK、雅可比组装、LM 迭代更新与块内候选筛选全部在单一 kernel 内完成。本文在 NVIDIA GeForce RTX 4060 Laptop GPU 上对方法进行了全面的实验验证和微架构分析。

本文方法的应用场景界定为: 批量目标数 N 在数百量级 ($N \leq 1000$)、机械臂构型固定为 6-DOF 的实时规划前端。在此场景下, GPU 调度开销是主要瓶颈; 当 N 继续增大时, 计算吞吐将受限于 FP64 管线发射延迟而非调度延迟, 届时混合精度加速成为必要路径, 本文将其列为后续工作的首要方向。

1 问题定义与评价协议

1.1 批量 IK 问题形式化

令 $q \in \mathbb{R}^6$ 为 UR10 六个旋转关节角, 正运动学 $T_{ee}(q) \in \text{SE}(3)$ 将关节角映射至末端 tool0 位姿。对目标 T^* , 定义平移误差 $e_p(q) = p_{ee}(q) - p^* \in \mathbb{R}^3$ 和旋转误差 $e_R(q) = \log(R^{*\top} R_{ee}(q))^\vee \in \mathbb{R}^3$ 。单目标 IK:

$$\min_{q \in [q_{\min}, q_{\max}]} \frac{1}{2} \|e_p(q)\|^2 + \frac{\alpha_R}{2} \|e_R(q)\|^2. \quad (1)$$

批量 IK 需对 N 个独立目标并行求解式 (1), 输出每目标的最佳候选解。

1.2 统一评价协议

- Strict 成功率: 同时满足 $e_p < 5 \text{ mm}$ 且 $e_R < 1^\circ$ 的目标比例;
- 全样本位置误差 p_{95} : 全部 N 个目标 (含失败) 位置误差的 95 分位数;
- 原始吞吐量: $N/\bar{t}_{\text{gpu}}$, $\bar{t}_{\text{gpu}}$ 为 CUDA event 测量的 GPU 流时间均值 (30 次重复)。

2 运动学结构驱动的 CUDA 小矩阵加速方法

本章按自底向上的顺序展开: 首先将运动学参数编码为 GPU 常量内存, 然后设计融合 FK 函数实现解析雅可比的高效组装, 接着给出无分支 LM 迭代算法, 最后将所有组件融合为单一 target-block 核函数。

2.1 运动学模型至 GPU 常量内存的编译期编码

UR10 运动学参数来源于官方 URDF 模型 [?]. 将以下编译期确定的信息编码为 CUDA `__constant__` 内存:

- 6 个关节轴单位向量 $a_i \in \mathbb{R}^3$ ($a_0 = Z, a_1 = Y, a_2 = Y, a_3 = Y, a_4 = -Z, a_5 = Y$);
- 6 个连杆偏移矩阵 $\text{origin}_i \in \text{SE}(3)$;
- 腕 3 至 tool0 固定变换 $T_{\text{wrist3_to_tool0}}$;
- 关节限位 $[q_{i,\min}, q_{i,\max}]$ 和障碍函数参数。

Ada Lovelace 架构 (SM 8.9) 提供 48 KB 常量缓存, 所有线程块通过广播总线零延迟访问。这些参数总计约 2 KB——不足缓存容量的 5%, 确保运行期间无常量缓存逐出。

2.2 融合 FK 函数与解析雅可比组装

数值差分雅可比需对每个关节执行正负扰动 FK, 每迭代 $6 \times 2 = 12$ 次额外 FK 调用。对平均 20 次 LM 迭代, 累计 ≈ 260 次 FK 调用——FK 成为计算瓶颈。本文设计的融合 FK 函数在单次前向传播中同步输出三个层次的几何量: 末端 tool0 位姿 T_{ee} 、各关节世界

位置 p_i 、各关节世界转轴 $z_i = R_{3 \times 3}^{(i)} \cdot a_i$ 。算法 1 给出了融合 FK 的计算流程。

表 1 算法 1 融合 FK 函数 (含坐标系输出)

```
// 输入: 6 维关节角向量
// 输出: 末端 tool0 位姿、关节世界位置、关节转轴
Input:  $q[0:5]$ 
Output:  $T_{ee}$ ,  $p[0:5][0:2]$ ,  $z[0:5][0:2]$ 
// 初始化 4×4 齐次变换矩阵为单位阵
 $T \leftarrow I_4$ 
for  $i = 0$  to 5 do
// 施加连杆偏移矩阵, 记录关节世界位置
 $T \leftarrow T \cdot \text{origin}[i]$ 
 $p[i] \leftarrow [T_3, T_7, T_{11}]$ 
// 提取当前累积旋转子块, 计算世界转轴向量
 $R_{3 \times 3} \leftarrow T_{0:2,0:2}$ 
 $z[i] \leftarrow R_{3 \times 3} \cdot \text{axis}[i]$ 
// 施加绕关节轴  $i$  的旋转
 $T \leftarrow T \cdot \text{Rodrigues}(\text{axis}[i], q[i])$ 
// 施加腕 3 至 tool0 固定变换
 $T_{ee} \leftarrow T \cdot T_{\text{wrist3\_to\_tool0}}$ 
return  $T_{ee}$ ,  $p$ ,  $z$ 
```

获得 (p_i, z_i) 后, 6×6 解析雅可比矩阵的组装仅需 6 次叉积运算 [?]:

$$J_{v,i} = z_i \times (p_{ee} - p_i), \quad J_{\omega,i} = z_i, \quad i = 0, \dots, 5. \quad (2)$$

与有限差分方案相比, 解析组装具有双重优势: (1) 计算量——无须重复 FK, 仅 6 次叉积 (各 6 FLOP) 替代 12 次 FK (各 ≈ 800 FLOP); (2) 精度——差分截断误差 $\mathcal{O}(10^{-6})$ vs 机器精度 $\mathcal{O}(10^{-16})$, 相差 10 个数量级。该精度优势对 Strict 阈值 ($5 \text{ mm}/1^\circ$) 判定具有直接影响: 差分方案的截断误差 (10^{-6}) 已接近或超过阈值量级, 可能将收敛解误判为失败; 解析方案的机器精度 (10^{-16}) 则完全消除这一风险。

2.3 无分支 LM 迭代算法

每个 Sobol 种子独立执行最多 60 次 LM 迭代。加权误差 $e = [e_p^\top, \alpha_R e_R^\top]^\top \in \mathbb{R}^6$ 与解析雅可比 $J \in \mathbb{R}^{6 \times 6}$ 构造正规方程:

$$(J^\top J + \lambda I) \Delta q = -(J^\top e + w_{\text{limit}} \nabla \Phi_{\text{limit}}), \quad (3)$$

其中 $\Phi_{\text{limit}} = \sum_{j=0}^5 \phi(q_j)$ 为关节限位二次障碍函数, $\phi(q_j) = \max(0, m - (q_j - q_{j,\min}))^2 + \max(0, m - (q_{j,\max} - q_j))^2$, margin $m = 0.087 \text{ rad}$ ($\approx 5^\circ$), $w_{\text{limit}} = 0.03$ 。关键设计: 阻尼因子 λ 摒弃传统 LM 的 acceptance-rejection 分支——若 $\rho > 0$ 则 $\lambda \leftarrow 0.5\lambda$, 否则 $\lambda \leftarrow 2\lambda$, $\lambda \in [10^{-6}, 0.5]$ ——新候选解总是被接受。这确保 32 线程经线内全部线程执行相同指令路径, 消除 warp

divergence。 6×6 正规方程的求解采用寄存器级高斯消元 (Gaussian elimination with partial pivoting) 在 6 次主元消去中完成, 所有中间变量保持在寄存器中, 零全局内存访问。

算法 2 给出了单个种子的完整 LM 迭代流程。

表 2 算法 2 单种子 LM 迭代 (无分支, 寄存器级小矩阵求解)

```
// 输入: Sobol 种子、目标位姿、最大迭代次数、初始阻尼
// 输出: 最优关节角、代价、成功等级
Input:  $q_{\text{seed}}$ ,  $T_{\text{target}}$ ,  $\text{max\_iter}=60$ ,  $\lambda_0=0.01$ 
Output:  $q_{\text{best}}$ ,  $\text{cost\_best}$ ,  $\text{success\_rank}$ 
// 初始化
 $q \leftarrow q_{\text{seed}}$ ;  $\lambda \leftarrow \lambda_0$ ;  $\text{cost\_best} \leftarrow \infty$ 
for  $\text{iter} = 1$  to  $\text{max\_iter}$  do
// 调用算法 1: 融合 FK 输出末端位姿 + 关节位置 + 转轴
 $(T_{ee}, p, z) \leftarrow \text{FusedFK}(q)$ 
 $e_p \leftarrow p_{ee} - p_{\text{target}}$ ;  $e_R \leftarrow \log(R^\top R_{ee})$ 
// 解析雅可比组装: 6 次叉积替换 12 次扰动 FK
for  $i = 0$  to 5 do
 $J_{0:2,i} \leftarrow z[i] \times (p_{ee} - p[i])$ 
 $J_{3:5,i} \leftarrow z[i]$ 
// 正规方程 + 寄存器级 6×6 高斯消元
 $H \leftarrow J^\top J + \lambda I$ ;  $g \leftarrow J^\top e$ 
 $\Delta q \leftarrow \text{GaussElim6} \times 6(H, -g - w \nabla \Phi)$ 
 $q_{\text{trial}} \leftarrow q + \Delta q$ ; clamp to  $[q_{\min}, q_{\max}]$ 
// 损失评估: 位姿代价 + 限位障碍
 $\text{cost} \leftarrow 0.5 \|e\|^2 + w_{\text{limit}} \Phi(q_{\text{trial}})$ 
 $\rho \leftarrow (\text{cost\_old} - \text{cost}) / \text{cost\_old}$ 
// 阻尼自适应: 无分支——总是接受 trial, 仅调节  $\lambda$ 
if  $\rho > 0$  then  $\lambda \leftarrow 0.5\lambda$  else  $\lambda \leftarrow 2\lambda$ 
 $q \leftarrow q_{\text{trial}}$ 
if  $\text{cost} < \text{cost\_best}$  then  $(q_{\text{best}}, \text{cost\_best}) \leftarrow (q, \text{cost})$ 
// 收敛判定
if converged( $\|e_p\| < 5 \text{ mm}$ ,  $\|e_R\| < 1^\circ$ ) then break
return  $(q_{\text{best}}, \text{cost\_best}, \text{success\_rank})$ 
```

2.4 Target-Block 融合映射: 单 Kernel 端到端执行

上述 LM 迭代对 N 个目标 $\times K$ 个种子独立执行, 共 $N \times K$ 个独立求解任务。传统方案的流水线为: 核函数启动 $\rightarrow$ 全局内存写回候选解 $\rightarrow$ CPU 后处理筛选 $\rightarrow$ 再次启动核函数——核函数启动次数为 $N \times K \times W + N$ (W 为权重级别数)。本文的 target-block 融合映射将全部工作压缩为单次核函数启动, 如算法 3 所示。

算法 3 的三个关键数据结构设计为:

(1) **Grid-Block 映射**。 $\lll N, 32 \ggg$ 启动配置将 N 个目标位姿一对一映射至 N 个线程块, 每块 32 线程中 lane 0-15 各承载一个 Sobol 种子 ($K = 16$), lane 16-31 空闲。这种映射使每个目标的计算完全局部化于

表 3 算法 3 Target-Block 融合核函数 (单 Kernel, 共享内存块内选择)

```
// 核函数签名与启动配置
Kernel: IK_LM_Multiseed_Block_Target
Grid: N blocks, Block: 32 threads
Constants: T_targets[N], seeds[N][K][6] (global)
Shared: s_cand[K][stride] (K=16, stride=16)
Output: best[N][kBestStride]

// 每个 Block 处理一个目标; 16 条 lane 各执行一个 Sobol 种子
tid ← blockIdx.x; lane ← threadIdx.x
if lane < K then
// 加载种子, 调用算法 2 执行 LM 迭代
qseed ← seeds[tid][lane]
(qbest, cost, rank) ← LM_Iter(qseed, T_targets[tid])
// 候选解写入共享内存 (列步长布局消除 Bank 冲突)
for j = 0 to 5 do s_cand[lane][j] ← qbest[j]
s_cand[lane][6..8] ← 目标平移分量
s_cand[lane][9] ← cost
s_cand[lane][10] ← rank
s_cand[lane][11] ← (near_limit ? 1 : 0)
// 同步全部 16 lane 的候选解写入
__syncthreads()
// 仅 lane 0 执行块内三级层次化选择
if lane = 0 then
best_idx ← 0
for k = 1 to K - 1 do
// 优先级 1: success rank
if s_cand[k][10] > s_cand[best_idx][10] then
best_idx ← k
else if equal then
// 优先级 2: near_limit (0 优于 1)
if s_cand[k][11] < s_cand[best_idx][11] then
best_idx ← k
// 优先级 3: pose cost 升序
else if equal ∧ s_cand[k][9] < s_cand[best_idx][9]
then best_idx ← k
// 最佳候选从共享内存写回全局内存
for j = 0 to 5 do best[tid][j] ← s_cand[best_idx][j]
best[tid][6] ← s_cand[best_idx][9] (pose cost)
best[tid][7] ← s_cand[best_idx][10] (success rank)
```

一个 SM 内——无跨块通信、无全局同步。

(2) 共享内存列步长布局。候选解数组 $s_cand[16][16]$ 采用 $kCandidateStride=16$ 的列优先存储 (C 语义下为 $[16][16]$ 行优先布局, 但访问模式为列优先: 相邻 lane 写入同一字段时地址偏移 $16 \times 8 = 128$ bytes = 4 个 Bank 宽度)。由于 GPU 共享内存共 32 个 Bank (每 Bank 4 bytes), 128 bytes 偏移恰好跨过 32 个 Bank 后回到同一 Bank 的同一 phase——但由于 32 与 16 互质, 相邻 lane 不会同时访问同一 Bank。PTX 汇编确认 Bank 冲突为零。

(3) 块内三级层次化选择。lane 0 扫描 16 个候选解的优先级: success rank (Strict > Medium > Loose > Fail) → near_limit 标志位 (0 优于 1) → pose cost 升序。该选择逻辑完全在共享内存和寄存器中完成, 零额外 kernel launch。

图 ??-?? 展示了线程映射架构和完整算法流程。

图 1 Target-Block 线程映射架构

2.5 Kernel Fusion 的调度开销分析

表 ?? 给出了 Kernel Fusion 前后的核函数启动次数对比。传统多 kernel 方案中, 逐种子逐权重分别启动核函数导致累计启动延时随 $N \times K \times W$ 线性增长; 本文方案将其压缩至常数 1, 使调度开销从毫秒级降至微秒级 ($< 0.01\%$ GPU 流时间)。这一结构性优势在 $N \leq 1000$ 的低批量场景中尤为突出——cuRobo 虽通过 CUDA Graph 缓解了部分启动延迟, 但其多阶段流水线 (粒子初始化、评估、L-BFGS 更新) 在 $N = 100$ 时固定开销占比仍然显著。

表 4 Kernel Fusion 前后的调度开销对比

	传统多 Kernel	本文 Kernel Fusion
启动次数	$NKW+N$	1
$N=100$ 累计延迟	>1 ms	≈ 7 μ s
占 GPU 时间比例	$\sim 10\%$	$< 0.01\%$

3 实验设置

所有实验在单块 NVIDIA GeForce RTX 4060 Laptop GPU 上进行 (Ada Lovelace, SM 8.9, 24 组 SM, 8 GB GDDR6, FP64 理论峰值 ≈ 0.18 TFLOPS)。软件栈: CUDA Toolkit 13.3 (nvcc V13.3.33)、驱动 610.43.02、GCC 11.4.0、-O3 -DCMAKE_BUILD_TYPE=Release。求解器参数: variant = opt4c_block_target、precision =

图 2 求解器完整算法流程

fp64、limit_gradient = analytic、 $K = 16$ (Sobol 序列 [?])、预热 10 次、重复测量 30 次。所有 GPU 时间由 CUDA event API 测量 (cudaEventElapsedTime)，排除首轮预热和内存分配时间。

目标位姿采用“随机关节角 $\rightarrow$ FK”策略生成，确保物理可达。固定随机种子 42，生成 $N = 100, 200, \dots, 1000$ (步长 100，共 10 个规模) 的目标位姿 raw 文件 ($[N, 16]$ double，行优先 4×4 齐次矩阵)。cuRobo 对比使用默认 cuRobo-Graph 模式 (CUDA Graph 开启、碰撞检测关闭、单种子 num_seeds=1)，所有结果经同一外部 URDF FK 评估管线复评。

4 实验结果与分析

4.1 静态批量 IK 综合性能

表 ?? 给出 10 个规模的综合结果。全部规模通过单调性检查 (Loose SR $\geq$ Medium SR $\geq$ Strict SR)，零 NaN/Inf 异常。Strict 成功率 0.940–0.960， $N \geq 500$ 时全样本位置误差 p_{95} 均低于 5 mm Strict 阈值 ($N = 300$ 时个别失败样本拉高 p_{95} ——小样本下单个失败对分位数影响更大)。平均迭代 19–21 次，近限位比例 $< 1.0\%$ 。

吞吐量增长模式。 $N = 100 \rightarrow 1000$ 吞吐量仅提升 21% ($1.50 \rightarrow 1.82 \times 10^4$)，符合典型的固定开销分摊曲线——该趋势可由 Amdahl 定律刻画：lane 0 串行候选选择 (≈ 0.005 ms/target) 构成不可并行的串行分量。GPU 流时间与 N 严格线性 ($R^2 > 0.999$ ，斜率 0.0549 ms/target)，不随批量增大而退化——这是 target-block 映射结构确定性的直接验证。

GPU 时间组成 ($N = 1000$)。核函数执行占 GPU 流时间的 99.76% (54.74 ms)，H2D 传输 (目标位姿 + 种子上传) 0.550 ms (0.20%)，D2H 传输 (最佳解回读) 0.097 ms (0.04%)，核函数启动约 0.007 ms ($< 0.01\%$)。这一定量分解验证了 Kernel Fusion 消除调度开销的有效性：在多 kernel 方案中，仅核函数启动累计即可达毫秒量级。

图 ??–?? 给出了 CUDA OPT4C 与 cuRobo-Graph 在吞吐量、成功率和误差三个维度的密集对比。

图 3 批量吞吐量对比

图 4 Strict 成功率对比

图 5 全样本位置误差 p_{95} 对比

4.2 与 cuRobo-Graph 的系统级对比

表 ?? 给出与默认单种子 cuRobo-Graph 的对比。需说明：cuRobo 使用默认 num_seeds=1、外部种子置零的单种子配置；本方法使用 $K = 16$ Sobol 低差异序列。这并非质量上限对质量上限的对比，而是两种方法在各自默认配置下的系统级基准。

$N = 100$ 时本方法在吞吐量和精度上同时占优 (吞吐比 1.45:1)。 $N \geq 200$ 时 cuRobo 凭借 GPU 并行粒子群搜索反超， $N = 1000$ 时吞吐达 7.26×10^4 targets/s (约为本方法 4.0 倍)。然而，cuRobo Strict 成功率 (0.830–0.880) 明显弱于本方法 (0.940–0.960)。

cuRobo 误差的深层解读。表 ?? 同时报告了 cuRobo 全样本 p_{95} 和成功样本 p_{95} 。全样本 p_{95} 为 74–115 mm 完全由约 12%–17% 的失败样本尾部拉高——成功样本的位置误差 p_{95} 仅 0.1–0.3 mm，精度实际上高于本方法。两种方法的核心差异在于**是否能找到收敛解** (Strict 成功率)，而非**收敛后解的质量** (成功样本误差)。cuRobo 单种子配置的探索能力不足导致部分目标未能收敛，将整体 p_{95} 拉高了两个数量级。

注：cuRobo 使用默认单种子配置 (num_seeds=1)，其全样本 p_{95} (74–115 mm) 被失败样本尾部拉高，成功样本 p_{95} 仅 0.1–0.3 mm——求解器收敛后的精度高于本方法。两种方法的核心差异在成功率而非收敛精度。

4.3 种子数消融实验：多起点策略的决定性作用

为定量分离 Sobol 多起点策略对方法性能的贡献，将种子数从默认的 $K = 16$ 降至 $K = 1$ (仅保留每组 Sobol 序列的第一个种子)，保持其他全部参数不变。 $K = 1$ 时 16 条经线中仅 lane 0 执行 LM 迭代，其余 lane 空闲，块内候选选择退化为直接输出。表 ?? 给出了消融结果。

消融结果揭示以下关键事实：

(1) **成功率几乎完全来源于多起点策略。** $K = 16$ 时 Strict SR 稳定在 0.95 以上， $K = 1$ 时骤降至 0.45–0.52——降低约 44–51 个百分点。 $K = 1$ 的 p_{95} 误差高达 642–685 mm ($K = 16$ 时为 4.3–4.6 mm)，表明单种子时大量求解陷入不可接受的局部极小值，而 16 个 Sobol 低差异起点几乎完全消除了灾难性失败。

表 5 静态批量 IK 综合性能 ($N = 100 \sim 1000$, 步长 100)

N	GPU 时间/ms	吞吐量/(t · s ⁻¹)	Strict SR	p_{95} /mm	迭代次数	近限位
100	6.658	15019.0	0.960	4.38	21.0	0.010
200	13.724	14572.8	0.950	5.49	20.0	0.005
300	19.334	15516.5	0.940	27.04	20.0	0.007
400	24.090	16604.1	0.945	15.44	20.0	0.005
500	28.837	17338.9	0.954	4.34	20.0	0.004
600	33.677	17816.3	0.953	4.36	20.0	0.005
700	39.329	17798.8	0.953	4.78	19.5	0.006
800	44.421	18009.3	0.951	4.91	19.0	0.006
900	49.330	18244.6	0.951	4.90	19.0	0.007
1000	54.868	18225.5	0.954	4.56	19.0	0.007

表 6 与默认单种子 cuRobo-Graph 系统级对比

N	本方法吞吐/(t · s ⁻¹)	Strict SR	p_{95} /mm	cuRobo 吞吐/(t · s ⁻¹)	cuRobo SR	cuRobo 成功 p_{95} /mm
100	15019.0	0.960	4.38	10382.3	0.860	0.1
200	14572.8	0.950	5.49	20171.3	0.875	0.0
300	15516.5	0.940	27.04	30449.4	0.880	0.2
400	16604.1	0.945	15.44	36538.8	0.830	0.2
500	17338.9	0.954	4.34	43497.9	0.838	0.2
600	17816.3	0.953	4.36	49493.5	0.843	0.2
700	17798.8	0.953	4.78	55198.9	0.844	0.3
800	18009.3	0.951	4.91	61128.0	0.839	0.3
900	18244.6	0.951	4.90	64904.7	0.839	0.3
1000	18225.5	0.954	4.56	72624.3	0.840	0.3

表 7 种子数消融实验: $K = 16$ vs $K = 1$

N 吞吐比 $K=1/K=16$	$K=16$ SR	$K=1$ SR	$K=16$ p95/mm	寄存器限制/mm
				寄存器数/线程
100	0.960	0.450	4.38	160642.2
1.02				128
500	0.954	0.502	4.34	686.2
1.48				
1000	0.954	0.522	4.56	685.6
1.53				

(2) 速度-成功率交换率极高。 $K = 1$ 时吞吐量仅比 $K = 16$ 高 2%–53% ($N = 1000$ 时 2.79×10^4 vs 1.82×10^4), 但成功率却低了 83 个百分点。换言之, 以约 35% 的速度代价换取了超过 100% 的成功率提升——这是极其划算的工程权衡。

(3) 核函数启动次数与 K 无关。无论 $K = 16$ 还是 $K = 1$, 核函数启动次数恒为 1, 调度开销始终 $< 0.01\%$ GPU 流时间。这是本方法区别于传统多 Kernel 方案 (启动次数 $\propto N \times K \times W$) 的结构性优势。

(4) 与 cuRobo 单种子的公平对比。在同等单种子条件下, cuRobo Strict SR 为 0.830–0.880 (表 ??), 本方法 $K = 1$ 仅为 0.450–0.522——落后约 32–41 个百分点。这表明 cuRobo 的单粒子 L-BFGS 优化器具有更强的局部收敛能力。然而, 本方法通过 $K = 16$ 多起点策略将成功率推至 0.954, 反超 cuRobo 约 11–12 个百分点。两种方法代表了不同的设计范式: cuRobo 依赖”单粒子智能搜索”, 本方法依赖”硬件固化多起点”——前者适合大批量高吞吐场景, 后者在中小批量下提供成功率优先、延迟确定的求解保障。

4.4 微架构瓶颈的 PTX 级定量分析

为进一步探究吞吐量渐近收敛的微架构成因, 从 PTX 汇编层面进行定量分析。

寄存器压力与占用率。表 ?? 列出不同寄存器限制下主核函数的统计。无限制时每线程 194 个寄存器, 限制 160 时开始溢出 (216 bytes spill stores), 限制 128 时溢出显著增加 (568 bytes spill stores/740 bytes spill loads)。RTX 4060 Laptop GPU 每 SM 65 536 寄存器, 每经线 32 线程 $\times$ 194 寄存器 = 6 208 寄存器, 每 SM 最多驻留 $65536/6208 \approx 10.5$ 经线, 每块需 2 经线 (32 线程), 故每 SM 最多 5.3 块, 理论占用率 $10.5/24 \approx 44\%$ 。

44% 的中等占用率表明当前核函数为计算密集型而非延迟隐藏型: FP64 双精度管线的发射延迟是主要瓶颈, 而非全局/共享内存访问延迟。这意味着进一

表 8 不同寄存器限制下的 PTX 寄存器与溢出统计

寄存器限制/mm	寄存器数/线程	吞吐量/B	溢出存储/B	溢出加载/B
无限制	194	0	0	0
160	160	1534216	216	212
128	128	568	740	740
686.2		25724		

步优化方向不在数据搬运或 occupancy 提升, 而在于降低单迭代 FP64 指令延迟——例如将雅可比构造和 Hessian 累加迁移至 FP32 (Ada Lovelace 架构 FP32 吞吐量为 FP64 的 64 倍 [?]), 仅保留线性求解和收敛判定在 FP64。

共享内存 Bank 冲突分析。候选解存储采用 kCandidateStride= 16 的列步长布局。由于共享内存 Bank 数为 32 (每 Bank 4 bytes), 相邻经线 i 和 $i + 1$ 写入候选解同字段时地址偏移 $\Delta_{\text{addr}} = 16 \times 8 = 128$ bytes = 32×4 bytes——恰好跨越 32 个 Bank 后回到同一 Bank。此时两者访问同一 Bank 但不同 phase (4-byte 粒度), 不会触发 Bank 冲突。PTX 汇编确认共享内存 Bank 冲突计数为零, 总用量 2 064 bytes ($< 5\%$ of 48 KB/SM)。

浮点吞吐利用率估算。以 $N = 1000$ 为例: 每目标约 20 次迭代 $\times$ 每迭代约 1 650 FLOP = 33 000 FLOP/target, 总计 3.3×10^7 FLOP 在 54.9 ms 内完成, 实际 FP64 吞吐约 0.60 GFLOPS——仅为 RTX 4060 Laptop GPU 理论峰值 (约 180 GFLOPS) 的 0.33%。极低的浮点利用率源于大量标量寄存器操作、超越函数 (sin/cos/atan2) 和控制流指令——小型 6×6 矩阵运算中传统 FLOP/s 指标不适用, 这也是小矩阵 GPU 计算的普遍特征。

4.5 批量扩展性分析

图 ?? 展示了吞吐量和 GPU 时间随 N 的变化趋势。吞吐量从 $N = 100$ 的 1.50×10^4 渐近收敛至 $N = 1000$ 的 1.82×10^4 targets/s, 符合 Amdahl 定律预测——lane 0 串行候选选择构成不可并行化串行段。GPU 流时间严格线性 ($R^2 > 0.999$), 每目标固定开销 ≈ 0.055 ms, 用户无需对不同批量规模进行预扫描或参数调优即可精确预估吞吐量——这是定制 CUDA 线程映射相对于通用 GPU 调度器的核心优势。

图 6 批量扩展性分析

5 讨论

5.1 设计范式：硬件固化多起点 vs 单粒子智能搜索

消融实验（第 4.3 节）揭示了本方法成功的本质：其 0.95+ 的 Strict 成功率并非源于 LM 优化器本身（ $K=1$ 时仅 0.45–0.52），而是源于 $K=16$ Sobol 低差异序列在关节空间中的密集覆盖。这一定量事实将本方法的定位从“一个优化器的对比”提升为“一个并行求解范式的设计与验证”。

本方法代表的是**硬件固化多起点范式**：将计算资源（16 条经线 $\times$ 16 个种子）以编译期常量形式固化于单核函数内，无论种子数取值如何，核函数启动次数恒为 1，调度开销始终可忽略。cuRobo 代表的是**单粒子智能搜索范式**：依赖粒子群信息共享和 L-BFGS 的强局部收敛能力，以更少的种子数（默认 1）通过更智能的搜索路径达到收敛。两种范式的适用边界清晰：多起点范式在中小批量（ $N \leq 1000$ ）下提供成功率优先、延迟确定的求解保障；智能搜索范式在 $N \geq 500$ 时凭借粒子群并行获得更高吞吐量。二者并非优劣关系，而是面向不同工程需求的设计选择。

5.2 可推广性与加速方向

本方法的核心设计原则——编译期常量编码 + 单核函数固化 + 经线内无分支发散——不依赖于 UR10 的具体参数。对任意 n -DOF 串联机械臂（如 Franka Panda 7-DOF[?]），解析雅可比组装仍可通过融合 FK 获得，target-block 映射同样适用。两个明确的加速方向：（1）**混合精度**——FP32 执行雅可比和 Hessian（利用 Ada Lovelace FP32:FP64 = 64:1 吞吐比），FP64 保留于线性求解和收敛判定；（2）**种子 warm-start**——利用相邻目标在 SE(3) 中的空间邻近性共享收敛解，减少平均迭代次数。

6 结论

针对批量 IK 中单种子收敛概率不足与多 Kernel 调度开销过高的双重瓶颈，本文提出硬件固化的 Sobol 多起点并行 IK 加速方法。主要贡献：

（1）**Sobol 多起点 + Kernel Fusion 协同设计**：将 $K=16$ Sobol 种子固化于单核函数内，16 经线并行 LM 迭代 + 共享内存块内候选选择，核函数启动次数恒为 1。消融实验证实： K 从 16 降至 1 后吞吐量仅提升约 35%，但成功率从 0.954 暴跌至 0.522——多起点策略是成功率的决定性来源。

（2）**融合 FK + 解析雅可比**：单次 FK 输出 T_{ee} , p_i , z_i ，雅可比由 6 次又积组装，每迭代 FK 调用从 13 次降至 1 次，精度从 $\mathcal{O}(10^{-6})$ 提升至 $\mathcal{O}(10^{-16})$ 。

（3）**无分支 LM + 寄存器级 6×6 求解**：摒除 acceptance-rejection 消除经线内分支发散；高斯消元全程寄存器内完成。

（4）**系统级对比确立范式边界**：本方法 $K=16$ 时 Strict SR 0.940–0.960, $p_{95} < 5$ mm ($N \geq 500$)，成功率领先默认单种子 cuRobo 约 11–12 个百分点； $K=1$ 时落后 32–41 个百分点。两种方法代表不同范式——“硬件固化多起点”（成功率优先、延迟确定）vs“单粒子智能搜索”（高吞吐、适合大批量）。

（5）**PTX 微架构画像**：194 寄存器/线程， $\approx 44\%$ 理论占用率，共享内存 Bank 冲突为零。

综上，在当前边界条件（ $N \leq 1000$ 、固定 6-DOF、单消费级 GPU）下，本文方法以约 35% 的速度代价换取超过 100% 的成功率提升，确立了成功率优先、延迟确定的批量 IK 加速新范式。

后续工作：Nsight Compute 全量性能计数采样；FP32/F64 混合精度加速；扩展至 Franka Panda 7-DOF 验证可推广性。

参考文献

- [1] Siciliano B, Sciavicco L, Villani L, et al. Robotics: modelling, planning and control[M]. Springer, 2010.
- [2] LaValle S M. Planning algorithms[M]. Cambridge University Press, 2006.
- [3] NVIDIA. CUDA C++ programming guide[EB/OL]. [2026-06-10]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [4] Orocos Project. Kinematics and dynamics library (KDL)[EB/OL]. [2026-06-10]. <https://www.orocos.org/kdl.html>.
- [5] Beeson P, Ames B. TRAC-IK: an open-source library for improved solving of generic inverse kinematics[C]//IEEE-RAS International Conference on Humanoid Robots, 2015: 928–935.
- [6] Sundaralingam B, et al. cuRobo: parallelized collision-free robot motion generation[EB/OL]. arXiv: 2310.17274, 2023.

- [7] Nakamura Y, Hanafusa H. Inverse kinematic solutions with singularity robustness for robot manipulator control[J]. Journal of Dynamic Systems, Measurement, and Control, 1986, 108(3): 163-171.
- [8] Buss S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods[J]. IEEE Transactions on Robotics, 2004, 20(1): 1-18.
- [9] Universal Robots. Universal Robots ROS2 description[EB/OL]. [2026-06-10]. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [10] Levenberg K. A method for the solution of certain non-linear problems in least squares[J]. Quarterly of Applied Mathematics, 1944, 2: 164-168.
- [11] Marquardt D W. An algorithm for least-squares estimation of nonlinear parameters[J]. Journal of the Society for Industrial and Applied Mathematics, 1963, 11(2): 431-441.
- [12] Sobol I M. On the distribution of points in a cube and the approximate evaluation of integrals[J]. USSR Computational Mathematics and Mathematical Physics, 1967, 7(4): 86-112.
- [13] NVIDIA. CUDA C++ best practices guide[EB/OL]. [2026-06-10]. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.
- [14] NVIDIA. Nsight Compute user guide[EB/OL]. [2026-06-10]. <https://docs.nvidia.com/nsight-compute/>.
- [15] Chitta S, Sucan I, Cousins S. MoveIt![J]. IEEE Robotics & Automation Magazine, 2012, 19(1): 18-19.

作者简介

刘霄鹏 (2005-), 男, 本科, 主要研究方向为机器人运动学、GPU 并行计算。

E-mail: (预留)